LAPORAN PRAKTIKUM TEKNOLOGI IOT II



Disusun Oleh :

Muhammad Alan Al-faqih 25782074

Kelas 3C

Dosen :

Dr. Ir Septafiansyah Dwi Putra, S.T., MT.

Agus Ambarwari, S.Pd., M.Kom.

PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

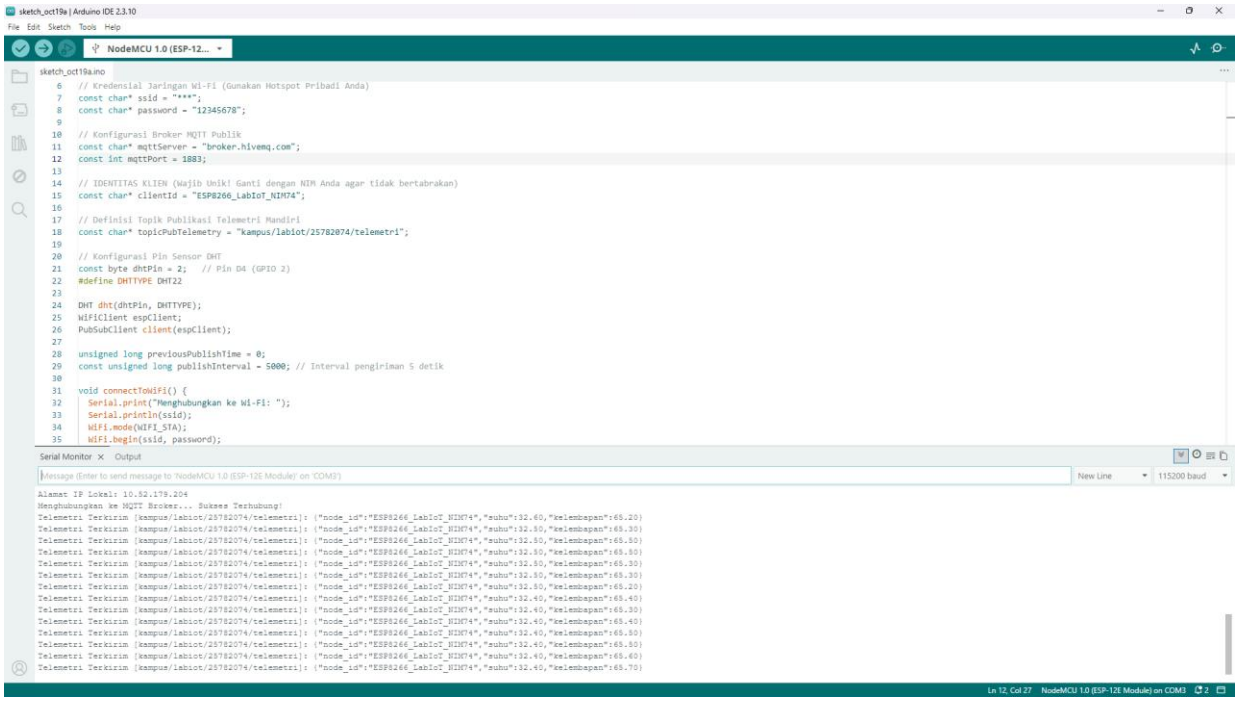
POLITEKNIK NEGERI LAMPUNG

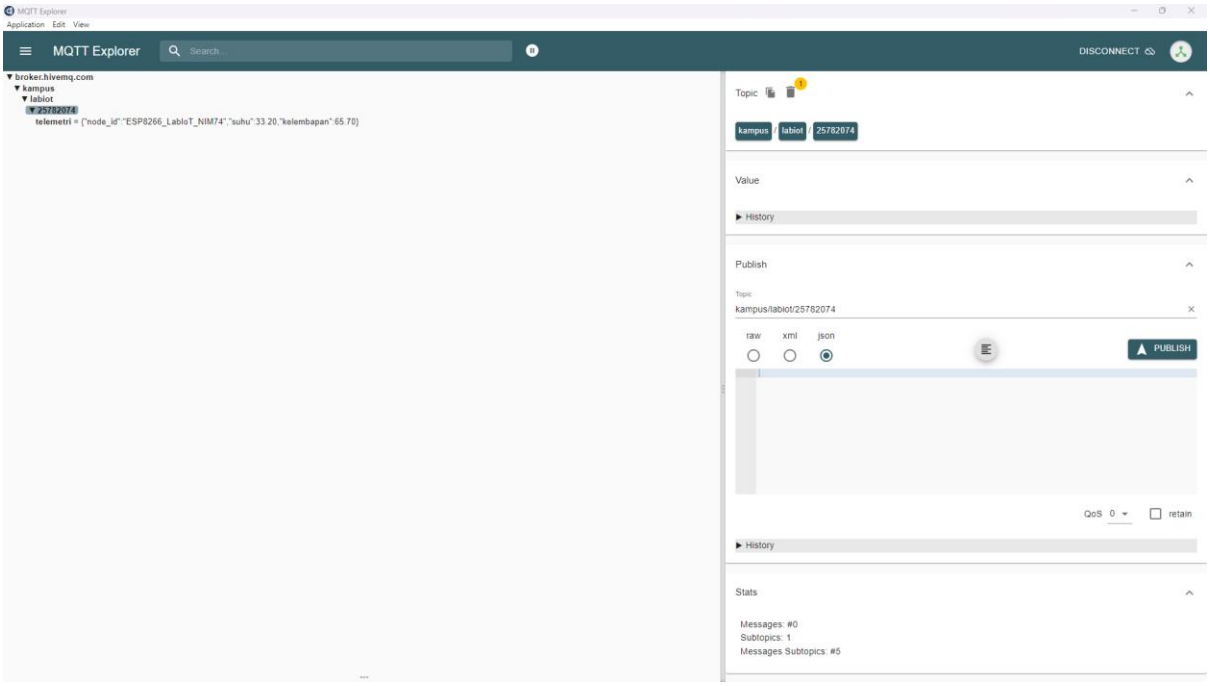
Praktikum 1: Publikasi Data Sensor Lingkungan (DHT) Berformat JSON

Pada praktikum ini, NodeMCU ESP8266 diprogram sebagai *Publisher*. Perangkat membaca data suhu dan kelembapan dari sensor DHT11/DHT22, mengemas parameter tersebut ke dalam dokumen JSON menggunakan ArduinoJson, lalu memublikasikannya setiap 5 detik ke broker publik broker.hivemq.com.

Langkah Kerja Rangkaian:

1. Pasang NodeMCU ESP8266 pada *breadboard*.
2. **Sensor DHT11/DHT22:** Hubungkan pin **VCC** ke **3V3**, pin **GND** ke **GND**, dan pin **Data** ke **D4 (GPIO 2)**.



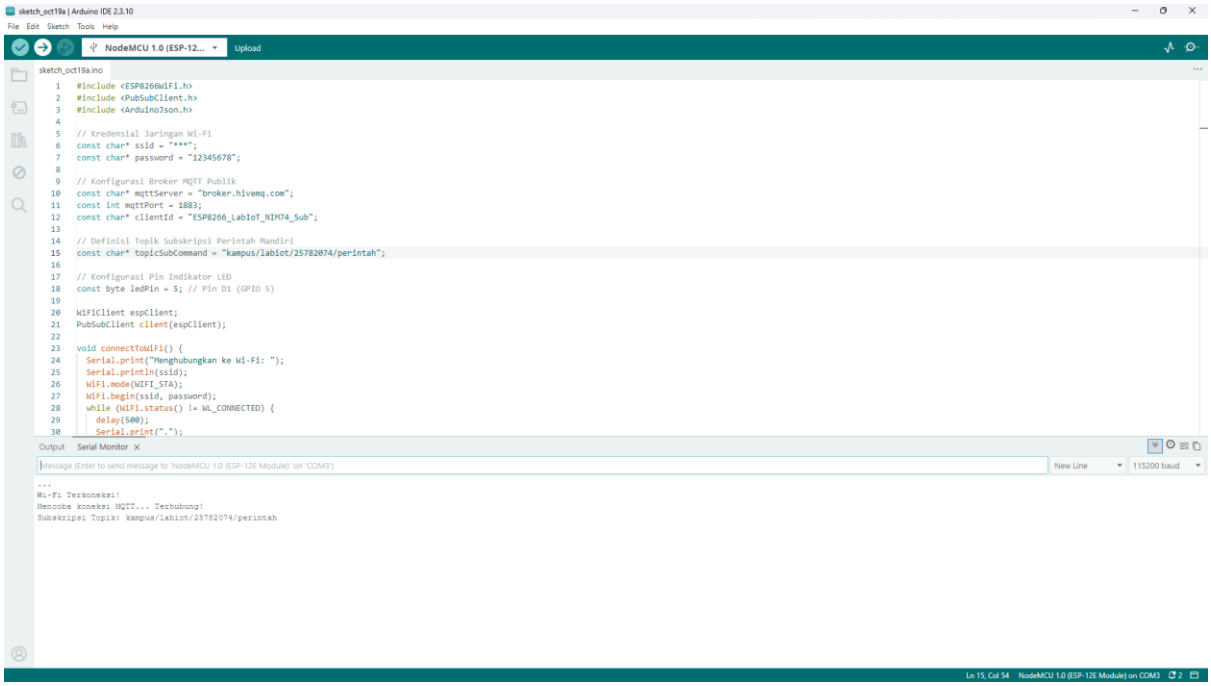

MQTT Explorer
Application Edit View
MQTT Explorer
Search...
DISCONNECT
broker.hivemq.com
kampus
labiot
25782074
telemetri = {"node_id":"ESP8266_LabIoT_NIM74","suhu":33.20,"kelembapan":65.70}
Topic
kampus / labiot / 25782074
Value
History
Publish
Topic
kampus/labiot/25782074
raw
xml
json
PUBLISH
QoS 0
retain
History
Stats
Messages: #0
Subtopics: 1
Messages Subtopics: #5

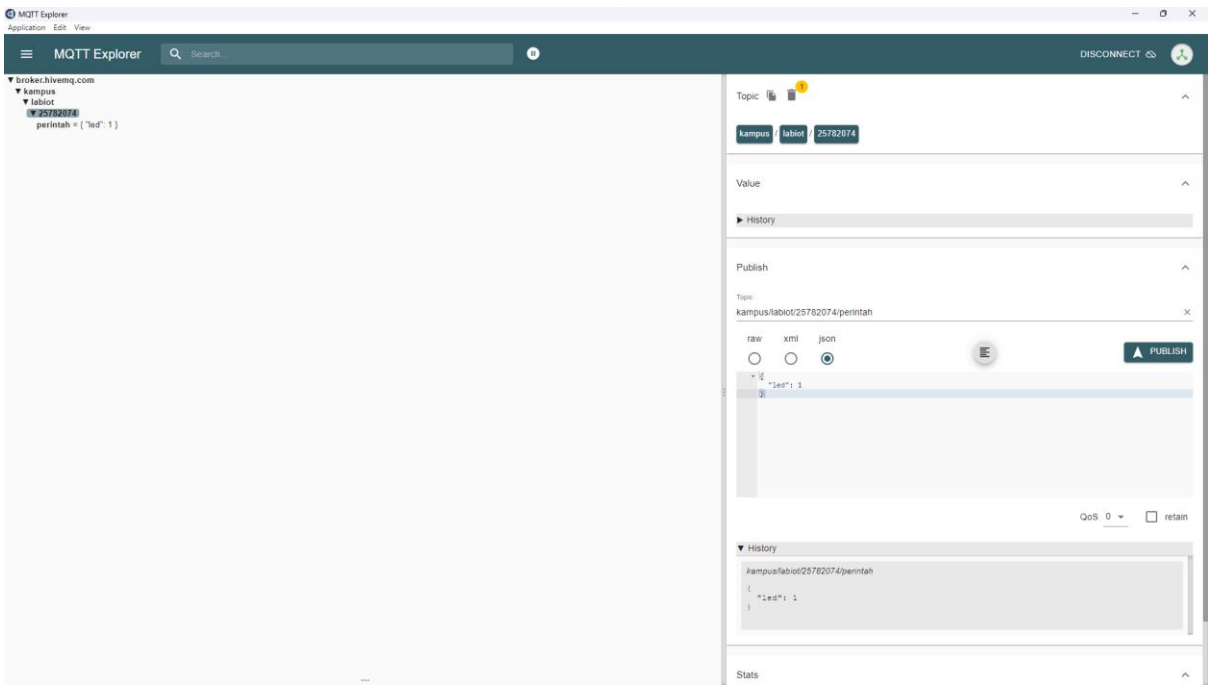
Praktikum 2: Subskripsi dan Kontrol Indikator LED Asinkron

Pada praktikum ini, NodeMCU ESP8266 diprogram sebagai *Subscriber*. Mikrokontroler mendaftarkan minatnya pada topik perintah (kampus/labiot/nim_anda/perintah). Saat pengguna luar (melalui aplikasi penguji MQTT) memublikasikan perintah berformat JSON, fungsi *callback* NodeMCU akan membedah payload dan menyalakan/mematikan lampu LED indikator secara seketika.

Langkah Kerja Rangkaian:

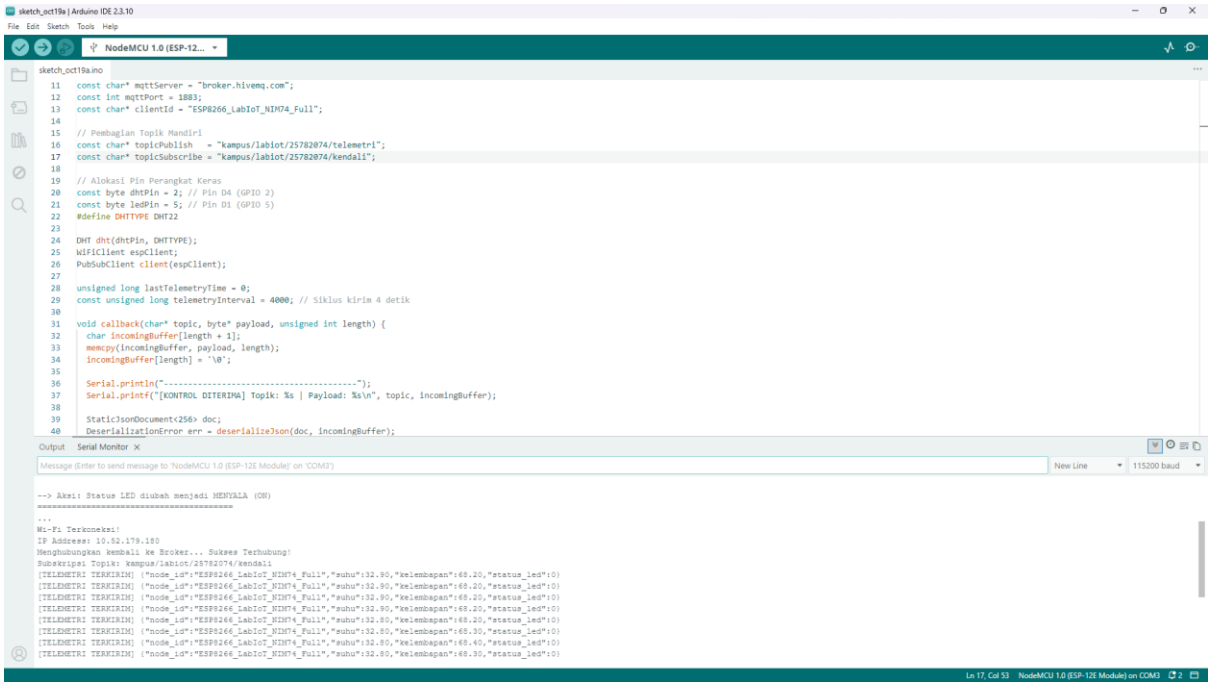
1. Pertahankan sensor DHT pada pin D4 dari Praktikum 1.
2. **LED Indikator:** Hubungkan kaki Anoda (kaki panjang) LED ke pin **D1 (GPIO 5)**, dan hubungkan kaki Katoda (kaki pendek) ke Resistor 220 Ωyang menuju ke **GND**.

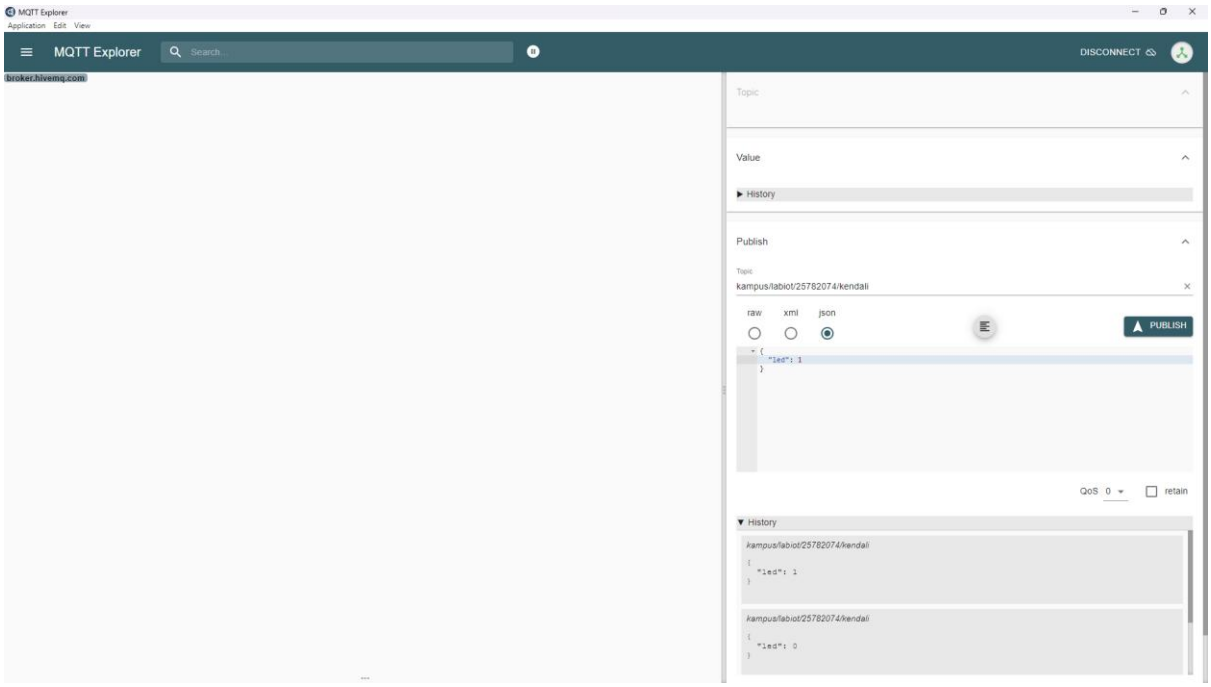




Praktikum 3: Sistem Terpadu Dua Arah (Full Duplex Telemetry & Control)

Pada praktikum ini, kita menyatukan kapabilitas Praktikum 1 dan Praktikum 2 ke dalam sebuah sistem terpadu (*Full-Duplex Telemetry & Control*). NodeMCU secara kontinu memublikasikan data telemetri sensor lingkungan (Suhu, Kelembapan, dan Status LED fisik) ke Broker setiap 4 detik, sembari tetap responsif mendengarkan dan mengeksekusi instruksi kontrol LED dari topik perintah kapan saja.





Latihan 1 (Analisis Arsitektur & Perbandingan Protokol):
Pada Modul 3, kita menggunakan protokol HTTP untuk membaca dan mengontrol perangkat. Jelaskan secara mendalam mengapa protokol MQTT jauh lebih unggul daripada HTTP dalam hal:

i. **Efisiensi penggunaan kapasitas *bandwidth* pada paket data seluler.**
ii. **Beban komputasi mikrokontroler (*CPU overhead*) saat menangani pertukaran data dua arah.**
iii. **Kemampuan pengiriman data secara simultan ke banyak perangkat penerima (*1-to-N distribution*).**

Jawab:

1. Efisiensi penggunaan bandwidth

MQTT lebih hemat bandwidth dibandingkan HTTP karena memiliki ukuran header yang kecil, minimal 2 byte. MQTT mengirimkan data melalui broker tanpa harus mengirimkan header HTTP yang besar pada setiap permintaan. Hal ini membuat MQTT lebih efisien untuk jaringan seluler dan perangkat IoT.

2. Beban komputasi mikrokontroler (CPU Overhead)

MQTT memiliki beban komputasi yang lebih ringan karena menggunakan model Publish-Subscribe dan pertukaran pesan yang sederhana. Mikrokontroler tidak perlu menangani permintaan HTTP berulang kali. MQTT juga mendukung komunikasi asinkron sehingga perangkat dapat menerima perintah tanpa menghentikan proses utama.

3. Pengiriman data ke banyak perangkat (1-to-N Distribution)

MQTT lebih unggul karena menggunakan broker sebagai pusat distribusi pesan. Publisher cukup mengirimkan data satu kali ke broker, kemudian broker meneruskannya kepada semua subscriber yang berlangganan pada topik tersebut. Dengan demikian, pengiriman data ke banyak perangkat menjadi lebih efisien.

Latihan 2 (Analisis Hierarki Topik & Karakter Wildcard):
Sebuah pabrik manufaktur memiliki 3 lini produksi (*line1*, *line2*, *line3*), di mana setiap lini memiliki 4 mesin (*m1*, *m2*, *m3*, *m4*), dan setiap mesin melaporkan metrik suhu, getaran, dan status. Struktur topiknya adalah pabrik/<line>/<mesin>/<sensor>. Tentukan ekspresi topik subskripsi (*subscription topic*) yang tepat menggunakan karakter *wildcard* (+ atau #) untuk:

i. **Memantau parameter suhu dari seluruh mesin di semua lini produksi.**
ii. **Memantau seluruh parameter sensor (suhu, getaran, status) khusus untuk mesin m1 di lini line2.**
iii. **Memantau seluruh lalu lintas data tanpa terkecuali dari seluruh lini dan seluruh mesin di pabrik tersebut.**

Jawab:

1. Memantau parameter suhu dari seluruh mesin di semua lini produksi

pabrik/+/+/suhu

Tanda + digunakan untuk menggantikan satu tingkat hierarki, yaitu lini produksi dan mesin.

2. Memantau seluruh parameter sensor khusus mesin m1 di line2

pabrik/line2/m1/#

Tanda # digunakan untuk menerima seluruh parameter sensor pada mesin m1 di line2, seperti suhu, getaran, dan status.

3. Memantau seluruh lalu lintas data dari semua lini dan mesin di pabrik

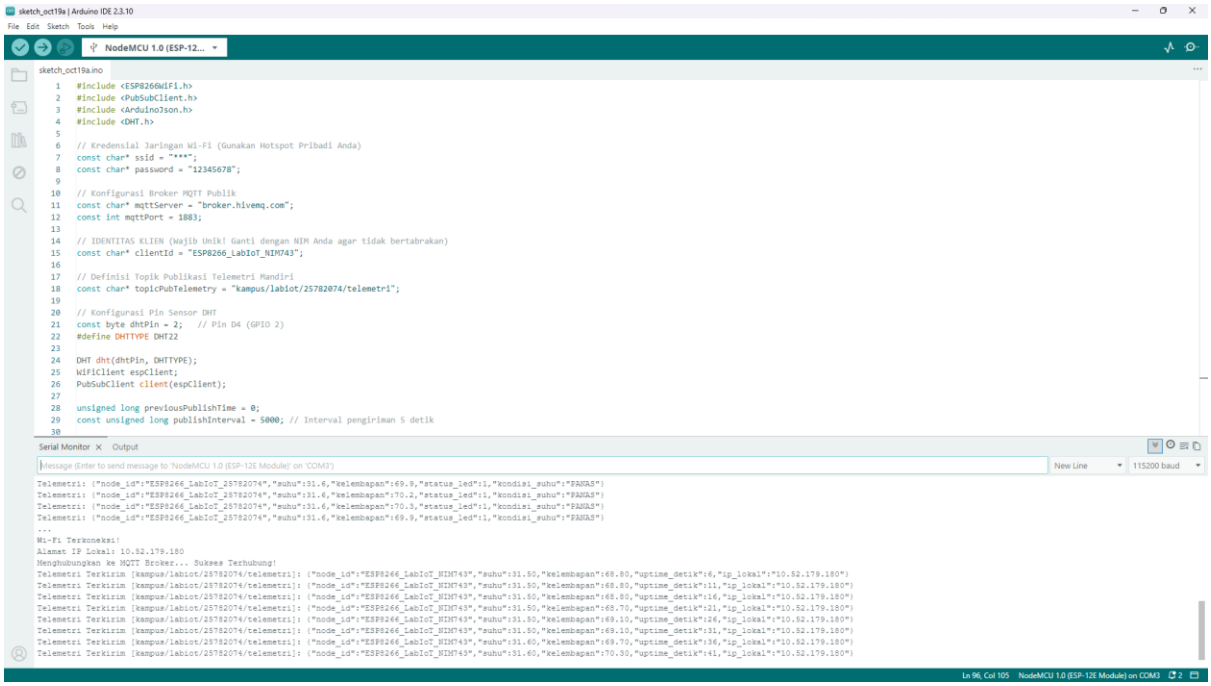
pabrik/#

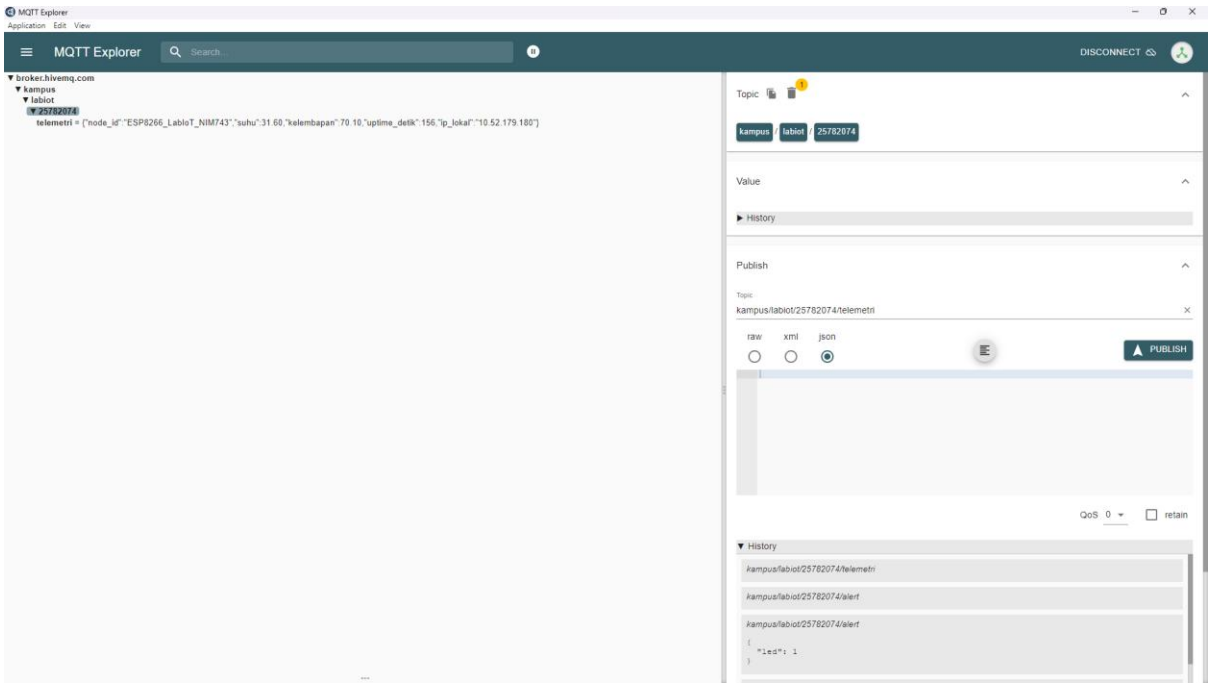
Tanda # digunakan untuk menerima semua pesan pada topik yang diawali pabrik/, termasuk seluruh lini, mesin, dan parameter sensor.

Latihan 3 (Implementasi Serialisasi Data Dinamis):
Modifikasi blok penyusunan JSON pada Praktikum 1 agar paket telemetri menyertakan dua parameter tambahan berikut:

i. "uptime_detik": Lama waktu aktif sistem dalam satuan detik (dihitung dari rumus millis() / 1000).

ii. "ip_lokal": Alamat IP lokal mikrokontroler dalam format string (WiFi.localIP().toString()).
Tuliskan baris modifikasi kode C++ yang Anda tambahkan pada fungsi pembentukan JSON!





Latihan 4 (Analisis Mekanisme Reconnect & Loop Blocking):
Pada Praktikum 3, di dalam fungsi loop() terdapat pemanggilan rutin client.loop().

i. Apa konsekuensi teknis yang terjadi terhadap responsivitas penerimaan perintah *callback* jika seorang pengembang menyisipkan instruksi delay(6000) di dalam fungsi loop()?

ii. Jelaskan bagaimana fungsi reconnect() bekerja memulihkan koneksi saat router Wi-Fi atau broker MQTT sempat terputus sesaat!

Jawab:

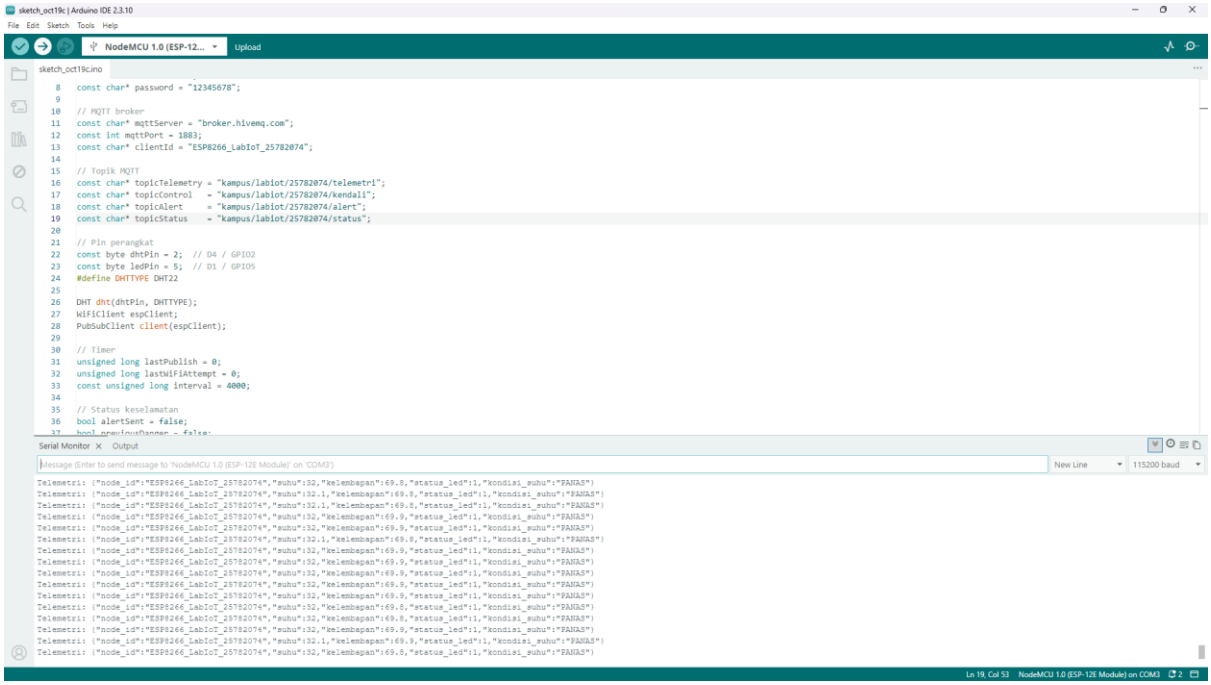
1. Konsekuensi jika ditambahkan delay(6000)

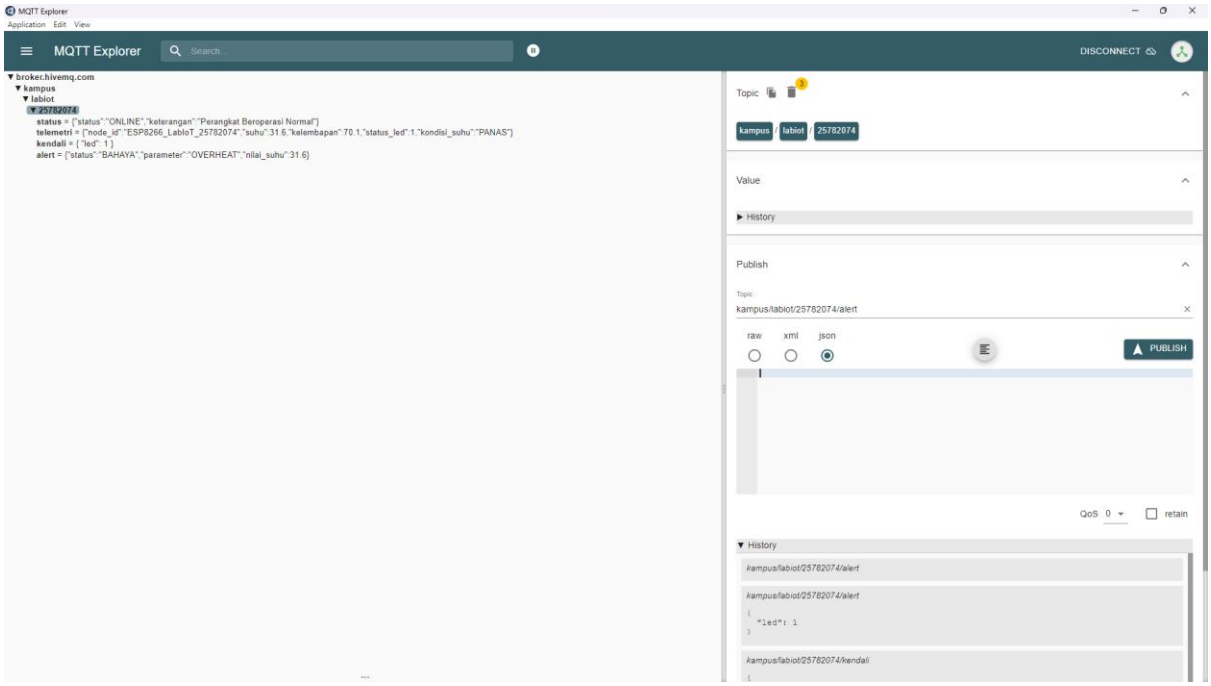
Perintah delay(6000) menghentikan proses program selama 6 detik. Akibatnya, pemanggilan client.loop() tertunda sehingga pesan MQTT tidak segera diproses dan fungsi callback terlambat menerima perintah. Hal ini membuat responsivitas sistem menurun.

2. Cara kerja fungsi reconnect()

Fungsi reconnect() memeriksa apakah koneksi MQTT masih aktif. Jika koneksi terputus, program akan mencoba menghubungkan kembali perangkat ke broker. Jika berhasil, perangkat kembali berkomunikasi dan dapat melakukan subscribe pada topik yang diperlukan. Jika gagal, program menunggu beberapa detik sebelum mencoba kembali hingga koneksi berhasil.

Tugas




MQTT Explorer
Application Edit View
MQTT Explorer
Search...
DISCONNECT
broker.hivemq.com
kampus
labiot
25782074
status = {"status":"ONLINE","keterangan":"Perangkat Beroperasi Normal"}
telemetri = {"node_id":"ESP8266_LabIoT_25782074","suhu":31.6,"kelembapan":70.1,"status_led":1,"kondisi_suhu":"PANAS"}
kendali = { "led": 1 }
alert = {"status":"BAHAYA","parameter":"OVERHEAT","nilai_suhu":31.6}
Topic
kampus / labiot / 25782074
Value
History
Publish
Topic
kampus/labiot/25782074/alert
raw
xml
json
PUBLISH
QoS 0
retain
History
kampus/labiot/25782074/alert
kampus/labiot/25782074/alert
{
"led": 1
}
kampus/labiot/25782074/kendali
{